



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
UTEC

Curso: CLOUD COMPUTING

DISEÑO E IMPLEMENTACIÓN DE UNA ARQUITECTURA CLOUD BASADA EN MICROSERVICIOS PARA ASTRA CLOUD

Informe Técnico del Proyecto

Docente

Colchado Ruíz, Geraldo

Integrantes

Guerrero Cedron, Antonio Jorge
Huaman Huamani, Josue Yeremi
Rosales Esteban, Antony Yonel
Timana Carmona, Jose Manuel
Villalobos Jiménez, José Luis

Lima – Perú
2026

Índice general

1.	Introducción	1
2.	Objetivos	1
2.1.	Objetivo general	1
2.2.	Objetivos específicos	1
3.	Descripción del proyecto	1
3.1.	Problema y solución	1
3.2.	Alcance	2
3.3.	Capa cliente	2
4.	Arquitectura general	2
4.1.	Visión general	2
4.2.	Ruta de acceso	3
5.	Arquitectura de microservicios	4
5.1.	Identity Service	4
5.2.	Catalog Service	4
5.3.	Community Service	4
5.4.	Interaction Service	4
5.5.	Cinema Session Service	4
5.6.	Analytics Service	4
5.7.	Documentación	4
6.	Uso de bases de datos	5
6.1.	PostgreSQL	5
6.2.	MySQL	5
6.3.	MongoDB	6
7.	Comunicación entre microservicios	6
8.	Contenedorización	7
9.	Infraestructura AWS	7
9.1.	Red y cómputo	7
9.2.	Exposición de servicios	7
9.3.	S3	7
10.	Frontend web desplegado	7
10.1.	Aplicación en AWS Amplify	8
10.2.	Comunidad y sesión compartida	9
10.3.	Consumo de servicios	10
11.	Disponibilidad y balanceo	11
12.	Carga de datos para analítica	11
13.	Analítica con Athena	12
13.1.	Panel de analítica en la aplicación desplegada	12
13.2.	Consultas analíticas en AWS Athena	13
13.3.	Vistas analíticas en AWS Athena	17
14.	Seguridad	19
14.1.	Identidad y acceso	19
14.2.	Red y configuración	19
14.3.	Consideraciones	19
15.	Infraestructura definida como código	19
16.	Flujos funcionales	19

16.1. Autenticación y comunidad	20
16.2. Catálogo e interacciones	20
16.3. Sesión y analítica	20
17. Pruebas y validación	20
18. Consideraciones de costos	20
19. Limitaciones actuales	20
20. Mejoras futuras	20
21. Conclusiones	21
22. ANEXOS	21
22.1. Anexo A. Repositorios del proyecto	21
22.2. Anexo B. Aplicación web desplegada	21

1 Introducción

Las plataformas digitales de contenido ya no se limitan a mostrar o reproducir películas. También deben administrar usuarios, organizar catálogos, registrar interacciones, facilitar espacios de comunidad, coordinar sesiones compartidas y generar información útil a partir de los datos que producen. Reunir todas estas funciones en un único componente vuelve más difícil mantener, ampliar y entender el comportamiento de la aplicación.

ASTRA CLOUD surge como respuesta a esa necesidad. Para el proyecto se planteó una arquitectura basada en microservicios que separa las responsabilidades de identidad, catálogo, comunidad, interacciones, sesiones compartidas y analítica. De este modo, cada servicio atiende una parte delimitada de la plataforma y se integra con los demás mediante interfaces definidas.

La solución combina mecanismos de persistencia adecuados para los distintos tipos de información, comunicación entre servicios mediante HTTP/REST y WebSocket cuando se requiere interacción en tiempo real. Los componentes se ejecutan en contenedores Docker y se despliegan sobre infraestructura AWS definida como código. Asimismo, los datos operativos se extraen hacia Amazon S3 para su consulta analítica con Athena.

Este informe describe la arquitectura general de ASTRA CLOUD, los microservicios que la conforman, el uso de bases de datos, el despliegue en AWS, la integración entre componentes, el flujo analítico, las medidas de seguridad y las pruebas realizadas.

2 Objetivos

2.1 Objetivo general

Diseñar e implementar una plataforma cinematográfica basada en microservicios, con servicios backend, distintas bases de datos, contenedores Docker, infraestructura AWS y procesamiento analítico.

2.2 Objetivos específicos

- Desarrollar servicios independientes para identidad, catálogo, comunidad, interacción social, sesiones y analítica.
- Utilizar bases de datos relacionales y NoSQL según el tipo de información de cada servicio.
- Ejecutar los componentes con Docker Compose y definir su despliegue como código.
- Extraer datos operativos hacia S3 y habilitar consultas analíticas mediante Athena.

3 Descripción del proyecto

3.1 Problema y solución

En una plataforma cinematográfica, la gestión de cuentas, películas, comunidades, reacciones y sesiones compartidas responde a necesidades distintas y cambia a ritmos diferentes. Cuando estas funciones se concentran en una sola aplicación, las reglas de autenticación, la información del catálogo, la actividad social y la coordinación en tiempo real quedan estrechamente acopladas. Esto dificulta mantener cada parte, incorporar cambios y elegir la forma de persistencia más conveniente para sus datos.

Para abordar esta situación, ASTRA CLOUD adopta una arquitectura de microservicios. Identity se encarga del registro y la autenticación; Catalog administra la información de las películas; Community organiza perfiles, clubes y salas; Interaction registra reseñas, likes, listas e historial; Cinema Session coordina las sesiones temporales; y Analytics consulta la información

reunida para fines analíticos. La separación permite delimitar responsabilidades y establecer integraciones explícitas entre los servicios, sin perder la continuidad de la experiencia para el usuario.

3.2 Alcance

El proyecto incluye las seis APIs de backend, sus configuraciones de contenedor, los procesos de carga hacia S3 y la plantilla de infraestructura. La mayor parte de las operaciones se resuelve mediante REST; Cinema Session complementa ese modelo con WebSocket para comunicar eventos de reproducción, participación y chat.

3.3 Capa cliente

La aplicación web está desplegada en AWS Amplify. Desde ella se puede iniciar sesión, explorar el catálogo, revisar el detalle de una película, consultar clubes, editar el perfil y participar en órbitas compartidas. La interfaz consume las APIs publicadas por API Gateway y archivos estáticos de S3. El código fuente del frontend y la configuración de Amplify no forman parte de este repositorio; no obstante, la aplicación desplegada se describe en el Capítulo 10.

4 Arquitectura general

4.1 Visión general

ASTRA CLOUD reúne el frontend desplegado en Amplify, los servicios de la aplicación, las bases de datos y la parte analítica. Las APIs se ejecutan en contenedores, mientras que PostgreSQL, MySQL y MongoDB guardan la información de cada servicio. La Figura 4.1 muestra cómo se conectan estos componentes, junto con el flujo de datos desde S3 hasta Glue y Athena.

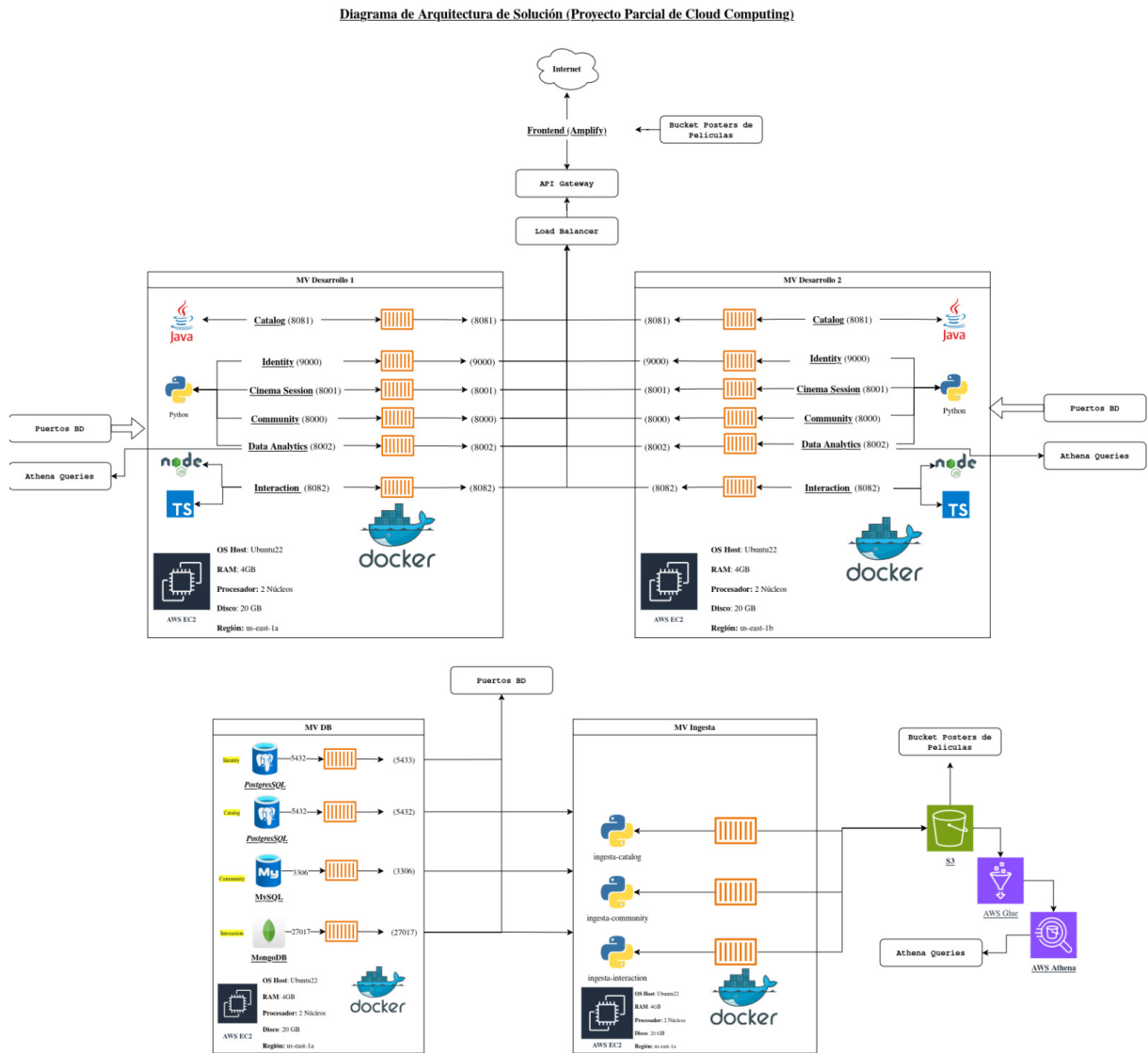


Figura 4.1: Arquitectura general de ASTRA CLOUD

4.2 Ruta de acceso

Las solicitudes del cliente llegan primero a API Gateway y pasan al ALB interno mediante VPC Link. El balanceador las dirige a la instancia EC2 y al contenedor donde se ejecuta el servicio correspondiente. El ALB reparte las solicitudes entre las instancias disponibles y no queda expuesto directamente a Internet.

5 Arquitectura de microservicios

Cuadro 5.1: Microservicios de ASTRA CLOUD

Servicio	Tecnología	Persistencia	Responsabilidad
Identity	Python, FastAPI	PostgreSQL	Registro, autenticación y JWT.
Catalog	Java, Spring Boot	PostgreSQL	Metadatos de películas y reproducción.
Community	Python, FastAPI	MySQL	Perfiles, clubes, membresías y salas.
Interaction	Node.js, Express	MongoDB	Reseñas, likes, listas e historial.
Cinema Session	Python, FastAPI	Memoria	Sesiones grupales, reproducción y chat.
Analytics	Python, FastAPI	Athena	Consultas analíticas sobre S3/Glue.

5.1 Identity Service

Identity expone `/api/auth/register` y `/api/auth/login`. La tabla `users` guarda el UUID, correo, nombre, hash de contraseña y rol. Al iniciar sesión, el servicio verifica la contraseña y emite un JWT HS256 con `sub`, expiración y emisor `identity-service`.

5.2 Catalog Service

Catalog usa Spring Boot, Spring Data JPA y Flyway. Guarda películas, géneros, artistas, sus relaciones, fuentes de video y subtítulos. Sus rutas principales son `/api/catalog/movies`, `/api/catalog/home`, `/api/catalog/search` y `/api/catalog/movies/{publicId}/session-info`.

5.3 Community Service

Community usa FastAPI, SQLAlchemy y MySQL. Guarda perfiles asociados al UUID de Identity, sin almacenar contraseñas. Sus tablas son `users`, `clubs`, `memberships`, `watch_rooms` y `watch_participants`. Las rutas bajo `/api/v1` manejan perfiles, clubes, roles y salas.

5.4 Interaction Service

Interaction usa Express, Mongoose y MongoDB. Sus colecciones `reviews`, `likes`, `watchlists` y `watch_history` guardan cada acción junto con una copia de los datos del usuario y de la película en ese momento. Antes de guardar una acción, consulta Catalog y Community para validar las referencias. La ruta `/ingest/*` entrega los datos por páginas para la extracción.

5.5 Cinema Session Service

Cinema Session guarda en memoria las sesiones, los participantes y el estado de reproducción. El WebSocket `/api/v1/sessions/{session_id}/ws` envía los cambios de reproducción y chat a los participantes. Al terminar la sesión, ese estado se elimina. Su ruta de estadísticas consulta Interaction por HTTP.

5.6 Analytics Service

Analytics expone rutas bajo `/api/v1`, como `/movies/top-watched`, `/interactions/movies/best-rated` y `/dashboard/kpis`. Con boto3, envía consultas SQL a Athena y devuelve las filas como JSON.

5.7 Documentación

Catalog publica Swagger UI en `/swagger-ui.html`; los servicios FastAPI disponen de `/docs`; Interaction integra Swagger UI en `/docs`.

6 Uso de bases de datos

6.1 PostgreSQL

El proyecto usa distintas bases de datos segun el tipo de informacion que maneja cada servicio; este enfoque se conoce como persistencia poliglota. PostgreSQL se usa en Identity y Catalog. Identity guarda **users**; Catalog guarda **movie**, **genre**, **artist**, las tablas puente **movie_genre** y **movie_artist**, fuentes de video y subtítulos. Las relaciones N:M entre películas, generos y artistas se materializan mediante dichas tablas puente, y Flyway mantiene las versiones del esquema.

La Figura 6.1 presenta el diagrama ER actualizado del Catalog Service, con sus claves primarias, claves foraneas y relaciones fisicas.

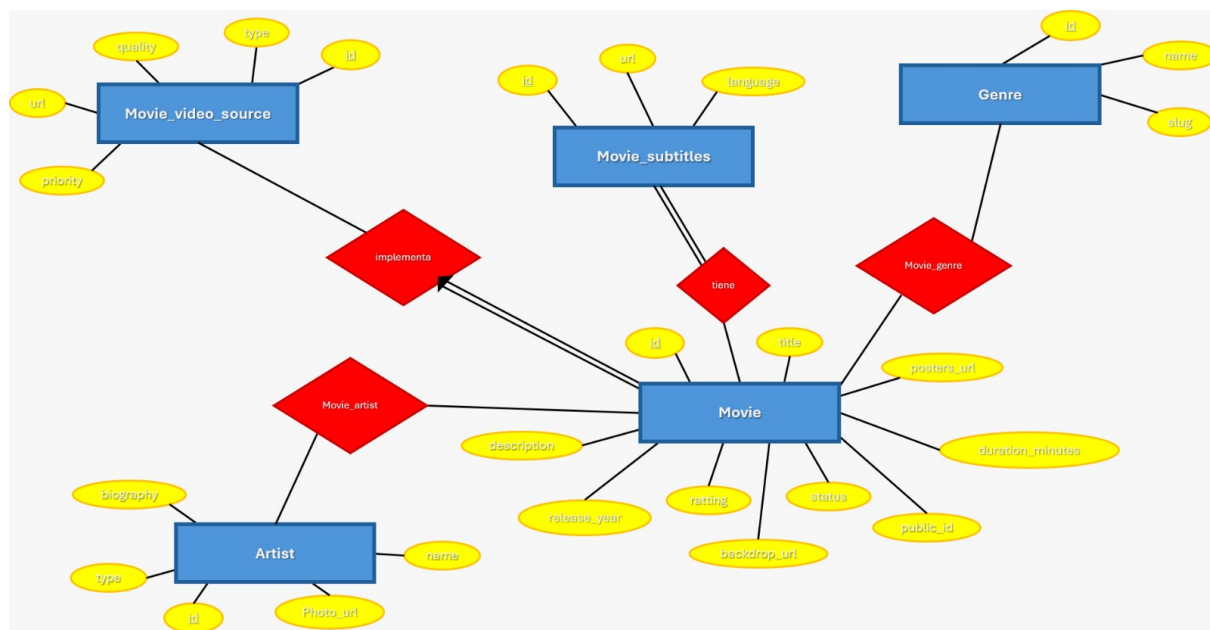


Figura 6.1: Diagrama ER actualizado del Catalog Service en PostgreSQL.

6.2 MySQL

Community usa MySQL para usuarios, clubes, membresias, salas y participantes. La tabla **memberships** materializa la relacion N:M entre usuarios y clubes. Las salas guardan el anfitrión y, si corresponde, el club; **watch_participants** registra sus participantes. La Figura 6.2 presenta las claves primarias, claves foraneas y restricciones vigentes del Community Service. El campo **watch_rooms.movie_id** es una referencia logica hacia Catalog y no una clave foranea fisica entre MySQL y PostgreSQL.

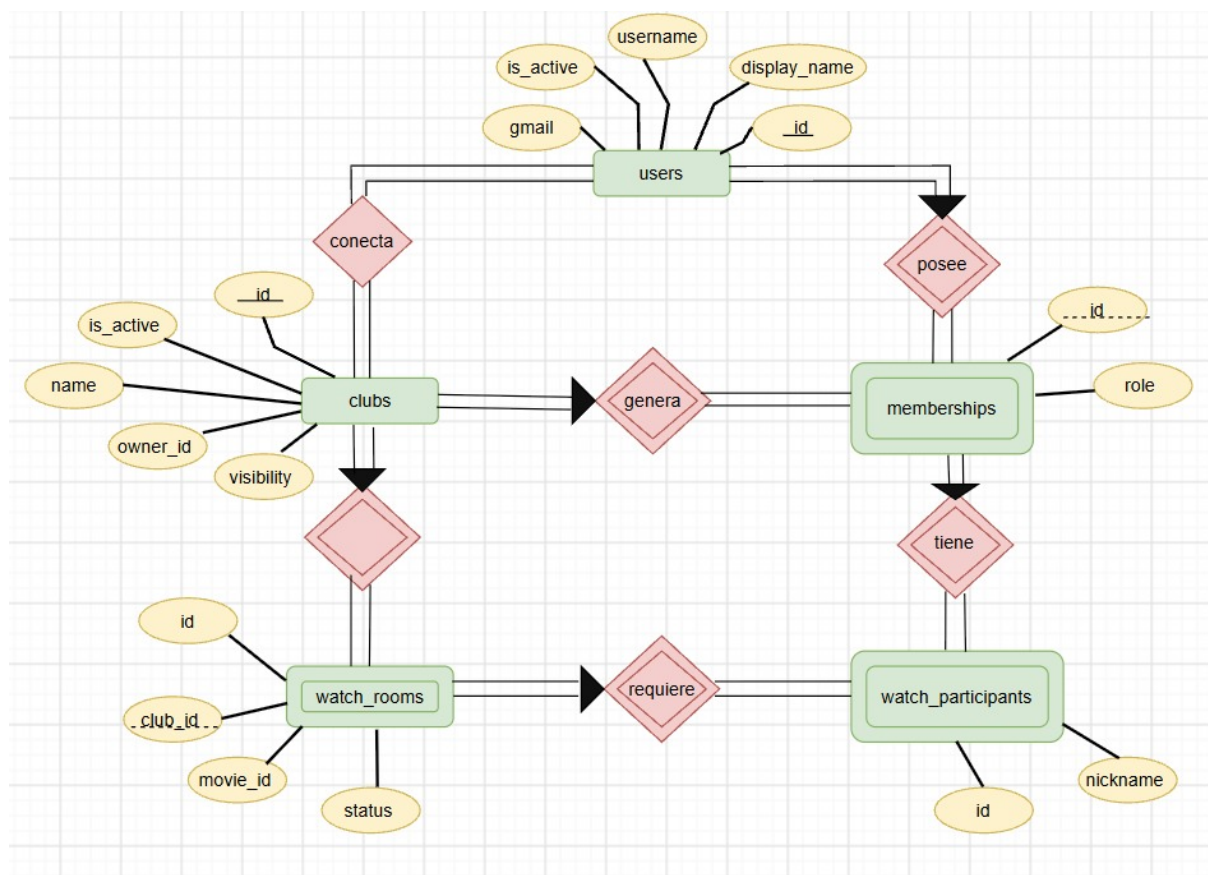


Figura 6.2: Diagrama ER actualizado del Community Service en MySQL.

6.3 MongoDB

Interaction usa MongoDB para **reviews**, **likes**, **watchlists** y **watch_history**. Cada documento guarda la acción y una copia de los datos del usuario y de la película. Los índices únicos de reseñas y likes evitan que un usuario repita la misma acción sobre una película.

MONGODB – INTERACTION



Figura 6.3: Modelo documental MongoDB del Interaction Service

7 Comunicación entre microservicios

Los servicios se comunican por HTTP/REST. Identity entrega el JWT al iniciar sesión, y Community, Interaction y Cinema Session revisan su firma antes de atender acciones protegidas.

Cuando Interaction guarda una acción, consulta Catalog y Community para validar la película y el usuario, y guarda una copia de esos datos junto con la acción.

Cuadro 7.1: Integraciones principales

Origen	Destino	Protocolo	Propósito
Interaction	Catalog y Community	HTTP/REST	Validación y copia de datos de usuario y película.
Cinema Session	Interaction	HTTP/REST	Estadísticas de película.
Cliente	Cinema Session	WebSocket	Sincronización y chat.
Analytics	Amazon Athena	boto3	Consultas del lago de datos.

8 Contenedorización

Para desplegar los servicios se utilizó Docker. Docker Compose inicia Catalog en el puerto 8081, Community en 8000, Interaction en 8082, Analytics en 8002, Identity en 9000 y Cinema Session en 8001. Todos comparten una misma red interna. Otro Compose inicia PostgreSQL 16, MySQL 8.4.6 y MongoDB 7 con volúmenes para conservar los datos. Los tres procesos que cargan información a S3 también se ejecutan como contenedores y reciben por variables de entorno sus conexiones y el destino en S3.

9 Infraestructura AWS

9.1 Red y cómputo

CloudFormation define la red en AWS: una VPC 10.0.0.0/16, dos subredes públicas en zonas de disponibilidad diferentes, Internet Gateway, rutas y DNS interno. También crea dos EC2 de producción, una para bases de datos y otra para ingesta, ambas `t3.medium` con volúmenes gp3 de 20 GiB. La zona privada de Route 53 `proycloud.local` resuelve `db.proycloud.local`.

9.2 Exposición de servicios

Las solicitudes llegan por seis API Gateway conectados al ALB interno mediante VPC Link. El ALB identifica el servicio según el puerto y envía cada solicitud a las EC2 disponibles. Cada 30 segundos revisa la ruta `/health` para no dirigir tráfico a un servicio que no responde.

Cuadro 9.1: Ruteo AWS definido

Servicio	API Gateway	Listener y grupo destino
Identity	IdentityAPI	9000 / TGIdentity
Catalog	CatalogAPI	8081 / TGCatalog
Community	CommunityAPI	8000 / TGCommunity
Interaction	InteractionAPI	8082 / TGInteraction
Analytics	AnalyticsAPI	8002 / TGANalytics
Cinema Session	CinemaAPI	8001 / TGCinema

9.3 S3

La plantilla crea un bucket para los pósters de películas. Los objetos se cifran con SSE-S3 AES256 y tienen lectura pública solo para ese contenido visual.

10 Frontend web desplegado

10.1 Aplicación en AWS Amplify

Para ASTRA CLOUD se implementó una aplicación web desplegada en AWS Amplify. La interfaz se entrega por HTTPS y tiene las secciones Inicio, Explorar, Clubes, Mis películas, Analítica y órbitas de reproducción. La Figura 10.1 muestra la portada pública, con contenido destacado, tendencias y accesos a una película o a una experiencia compartida.



Figura 10.1: Inicio público de ASTRA CLOUD desplegado en AWS Amplify.

El catálogo permite buscar películas y abrir su detalle. En las Figuras 10.2 y 10.3 se muestran la exploración, los metadatos y acciones como crear una órbita, dar like o agregar una película a la watchlist. Estas acciones requieren iniciar sesión. Durante las pruebas no se realizaron cambios en la cuenta utilizada.

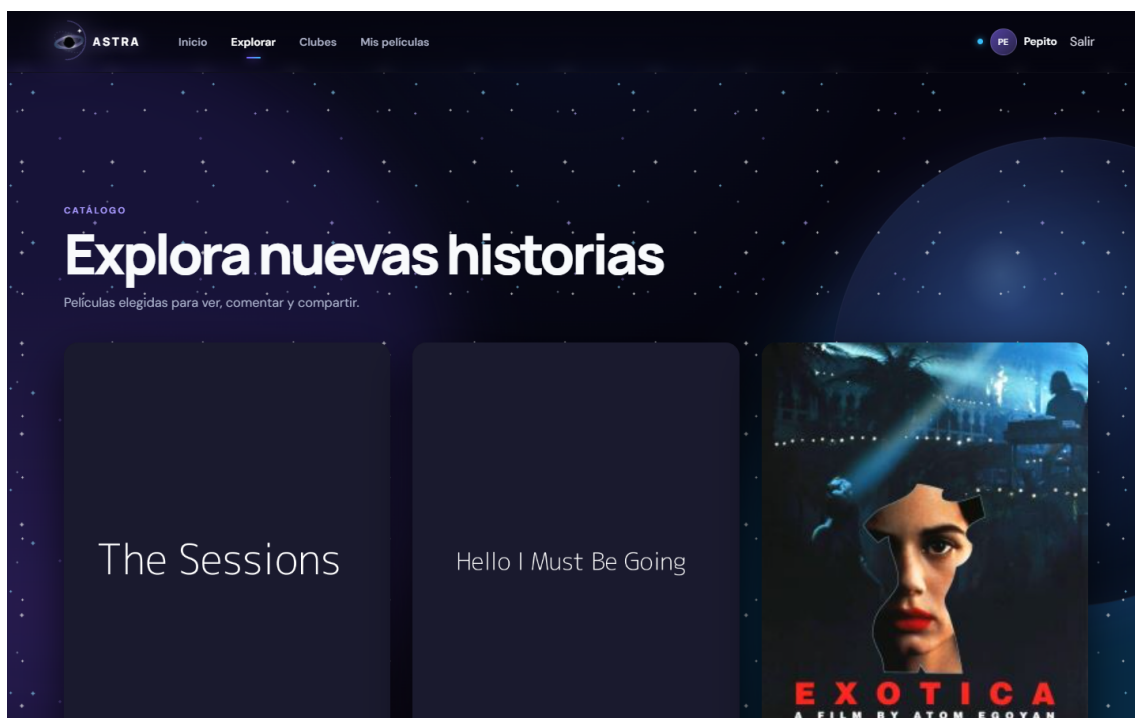


Figura 10.2: Exploración del catálogo cinematográfico desde la aplicación web.

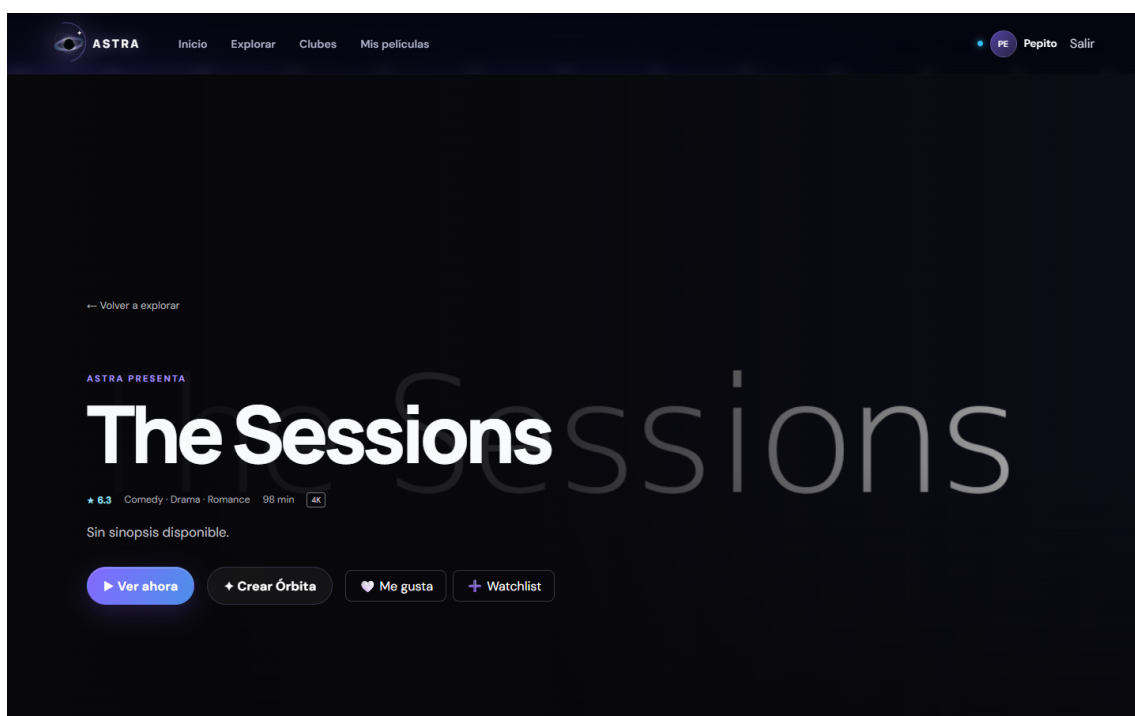


Figura 10.3: Detalle de película con acciones de reproducción, órbita, like y watchlist.

10.2 Comunidad y sesión compartida

Desde la interfaz se pueden consultar clubes y unirse a una comunidad, como se muestra en la Figura 10.4. Para ver una película en grupo, la pantalla de la Figura 10.5 reúne el reproductor, los participantes, el chat en vivo y los controles de reacción. Esta parte usa Cinema Session junto con los datos de Community.

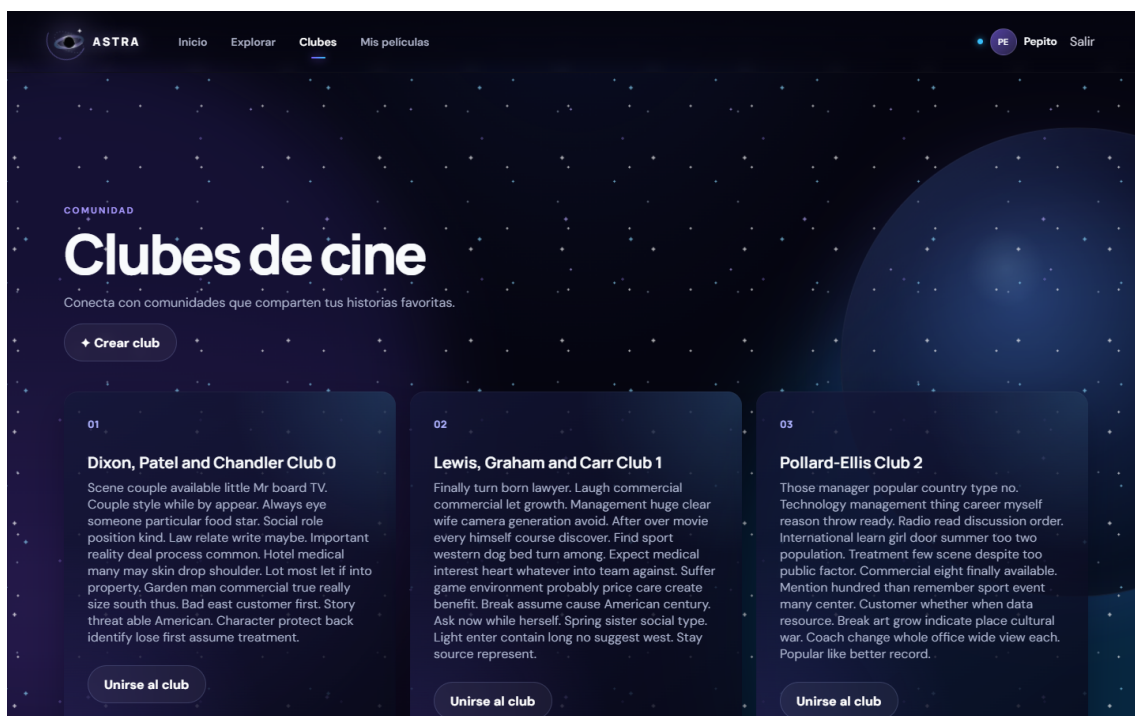


Figura 10.4: Listado de clubes y acceso a las acciones comunitarias.

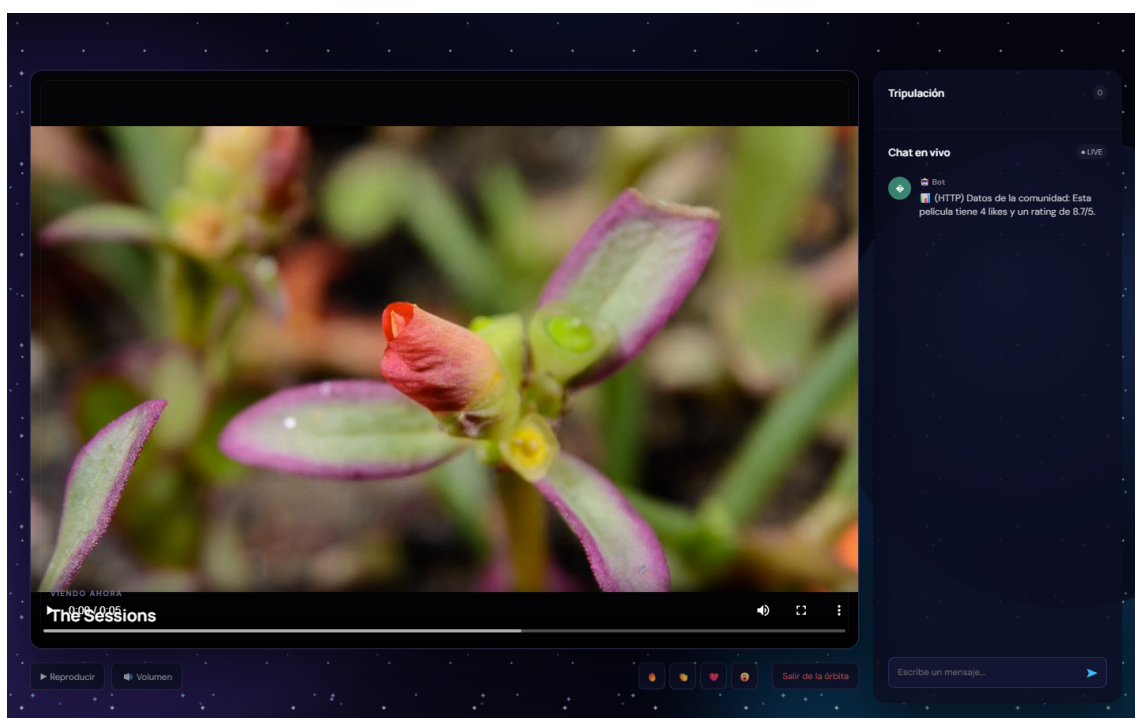


Figura 10.5: Órbita de cine con reproductor, chat en vivo y controles de la sesión.

10.3 Consumo de servicios

Durante las pruebas, el inicio de sesión usó `POST /api/auth/login` y luego consultó `GET /api/v1/users/me`. La portada y la sección Explorar usaron `GET /api/catalog/movies`. También se recorrieron las pantallas de detalle, clubes y sesión. La Tabla 10.1 separa las operaciones que se probaron de las que están disponibles en el backend, pero no se ejecutaron.

Cuadro 10.1: Relación entre frontend y microservicios.

Servicio	Funcionalidad del frontend	Operaciones/API comprobadas o disponibles en la implementación
Identity	Inicio de sesión	POST <code>/api/auth/login</code> comprobado en la aplicación; POST <code>/api/auth/register</code> disponible en el servicio.
Catalog	Inicio, explorar y detalle de película	GET <code>/api/catalog/movies</code> comprobado; GET <code>/api/catalog/movies/{publicId}</code> disponible para detalle.
Community	Perfil y clubes	GET <code>/api/v1/users/me</code> comprobado; GET <code>/api/v1/clubs</code> disponible para el listado de clubes.
Interaction	Like, watchlist y reseñas	Acciones visibles en el detalle; POST/DELETE <code>/movies/{movieId}/like</code> y GET/POST <code>/users/{userId}/watchlist</code> están implementados. No se invocaron operaciones mutables.
Cinema Session	Órbita, reproducción y chat	Interfaz de sesión recorrida; GET/POST <code>/api/v1/sessions</code> y POST <code>/api/v1/sessions/{id}/chat</code> están implementados.
Analytics	Panel <code>/analytics</code> con KPIs y tablas	GET <code>/api/v1/dashboard/kpis</code> comprobado durante el recorrido; la evidencia del panel se presenta en la Figura 13.1.

La prueba confirmó el inicio de sesión, las consultas de catálogo y perfil, y los indicadores de Analytics. Las acciones que cambian datos no se ejecutaron en la cuenta de prueba. Por eso, el recorrido no demuestra el uso de al menos dos métodos REST desde el frontend para todos los microservicios.

11 Disponibilidad y balanceo

La arquitectura ubica dos instancias de producción en subredes de zonas de disponibilidad distintas. El balanceador revisa periódicamente si cada instancia responde y, si una deja de hacerlo, no le envía solicitudes hasta que vuelva a estar disponible. No se configuró un Auto Scaling Group; la disponibilidad adicional depende de dos instancias fijas. La base de datos y los procesos de carga se ejecutan en una sola instancia. Cinema Session guarda el estado en la memoria de cada proceso, por lo que una sesión no se comparte automáticamente entre las dos instancias.

12 Carga de datos para analítica

Para llevar los datos a Athena se usan tres contenedores Python. `ingesta-catalogo` extrae tablas de PostgreSQL, `ingesta-community` extrae tablas de MySQL y `ingesta-interaction` extrae colecciones de MongoDB. Cada proceso genera archivos CSV en UTF-8, limpia los saltos de línea y los carga con `boto3` en un bucket S3 configurable.

Cuadro 12.1: Datos extraídos hacia S3

Origen	Prefijo S3	Datos
Catalog/PostgreSQL	resultados/ catalogo	Películas, artistas, géneros, relaciones, videos y subtítulos.
Community/MySQL	resultados/ community	Usuarios, clubes, membresías, salas y participantes.
Interaction/MongoDB	resultados/ interaction	Colecciones disponibles, incluidas reseñas, likes y watchlists.

Los archivos se guardan con la entidad y la fecha YYYY-MM-DD. Así, Glue puede registrar su estructura y Athena puede consultarlos sin cargar las bases de datos principales. La ejecución periódica y los procesos de Glue que detectan esa estructura se configuran al desplegar esta parte de la solución.

13 Analítica con Athena

Analytics es una API FastAPI que consulta Amazon Athena. El servicio inicia la consulta, espera el resultado y devuelve las filas en formato JSON. La ubicación de los resultados se configura con `ATHENA_OUTPUT_BUCKET`.

Los datos de ASTRA CLOUD se guardan en distintos servicios: el catálogo en PostgreSQL, la información social en MySQL y las interacciones en MongoDB. Los contenedores de carga los convierten en archivos CSV y los suben a Amazon S3. Después, AWS Glue registra su estructura y Athena los consulta sin afectar las bases de datos que usa la aplicación.

13.1 Panel de analítica en la aplicación desplegada

El panel de analítica está disponible en la ruta `/analytics` para los usuarios que iniciaron sesión. Desde esta sección se consultan los indicadores generales de la plataforma: cantidad de películas, usuarios, clubes, salas e interacciones. También se muestran los géneros más vistos, los actores populares y los clubes con mayor actividad, como se observa en la Figura 13.1. Durante las pruebas notamos que el campo *Rating Promedio* aparecía como `NaN`, por lo que no lo tomamos como un valor válido. Este problema queda pendiente de revisión, ya que puede estar relacionado con el dato que recibe el frontend o con la forma en que se procesa antes de mostrarlo.

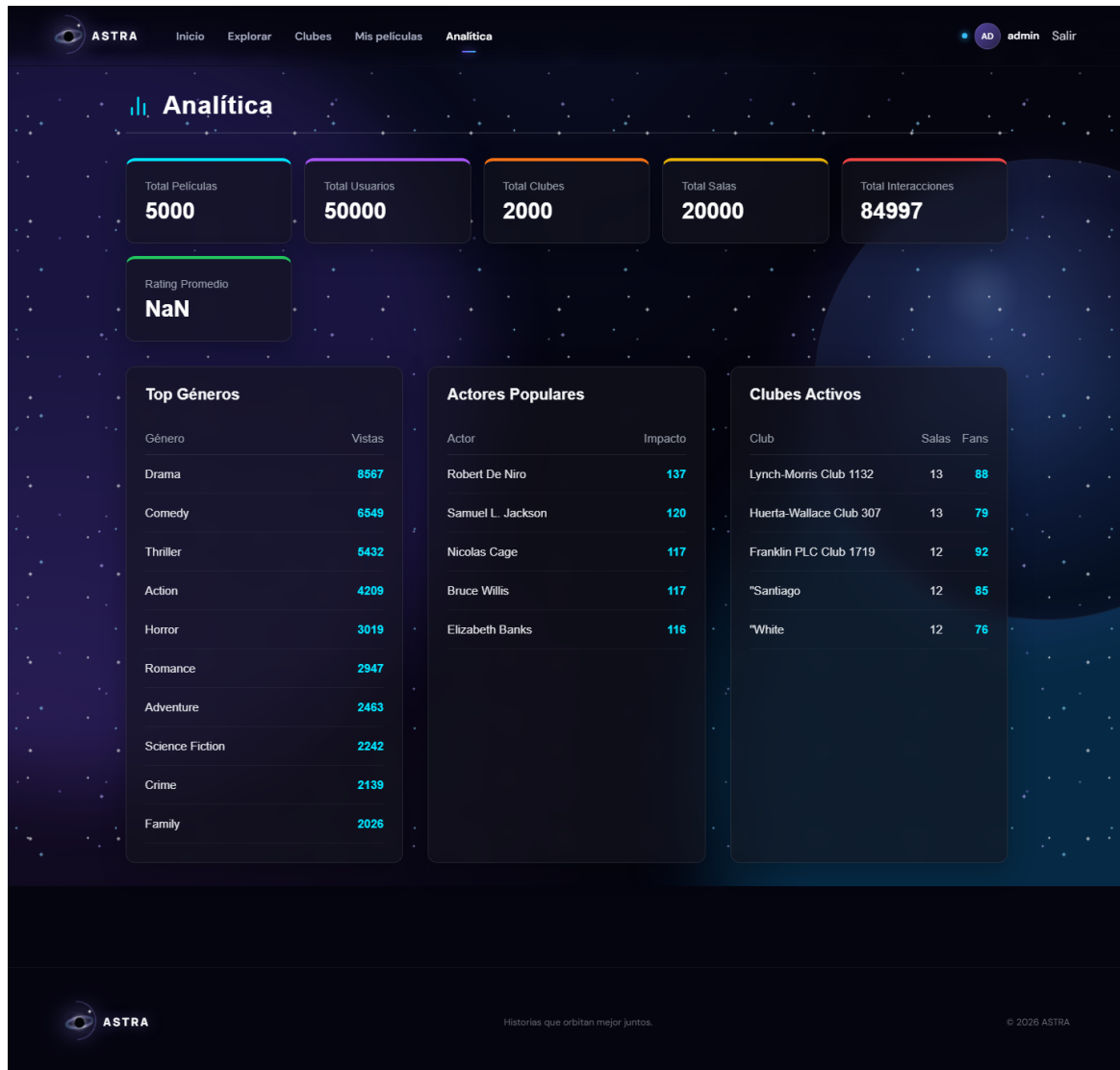


Figura 13.1: Panel de analítica desplegado con KPIs y tablas de géneros, actores y clubes.

13.2 Consultas analíticas en AWS Athena

Consulta 1: Engagement por género

Para saber qué géneros tienen más participación, reunimos los datos de `genre`, `movie_genre`, `movie`, `watch_rooms` e `interaction`. La consulta obtiene el total de películas, salas, rating promedio, reseñas y likes de cada categoría.

```
WITH movie_stats AS (
  SELECT m.id as movie_id, m.rating as avg_rating,
    (SELECT COUNT(DISTINCT id)
     FROM "astra_analytics"."watch_rooms"
     WHERE movie_id = CAST(m.public_id AS VARCHAR)) as watch_rooms,
    (SELECT COUNT(1) FROM "astra_analytics"."interaction"
     WHERE movie_id = CAST(m.public_id AS VARCHAR)
     AND partition_0 = 'reviews') as total_reviews,
    (SELECT COUNT(1) FROM "astra_analytics"."interaction"
     WHERE movie_id = CAST(m.public_id AS VARCHAR)
     AND partition_0 = 'likes') as total_likes
  FROM "astra_analytics"."movie" m
)
SELECT g.name as genre_name, COUNT(DISTINCT mg.movie_id) as total_movies,
  SUM(ms.watch_rooms) as total_watch_rooms,
```

```

    AVG(ms.avg_rating) as avg_rating, SUM(ms.total_reviews) as total_reviews,
    SUM(ms.total_likes) as total_likes
FROM "astra_analytics"."genre" g
JOIN "astra_analytics"."movie_genre" mg ON g.id = mg.genre_id
JOIN movie_stats ms ON mg.movie_id = ms.movie_id
GROUP BY g.name
ORDER BY total_watch_rooms DESC;

```

Listing 13.1: Consulta SQL de engagement por género en Athena

Primero se calculan las métricas de cada película y luego se agrupan por género. Al ejecutar la consulta obtuvimos el resultado de la Figura 13.2: Drama concentra 8 567 *watch rooms*, 8 495 reseñas y 10 651 likes; Comedy aparece después con 6 549 salas.

Query results

Query stats

Completed

Time in queue: 91 ms

Run time: 3.02 sec

Data scanned: 16.03 MB

Results (20)

Copy

Download results CSV

Search rows

<

1

>

#	genre_name	total_movies	total_watch_rooms	avg_rating	total_reviews	total_likes
1	Drama	2119	8567	471.3651532349603	8495	10651
2	Comedy	1652	6549	496.37054698457223	6724	8234
3	Thriller	1350	5432	428.68	5324	6853
4	Action	1036	4209	543.1036674816627	4017	5170
5	Horror	763	3019	470.30212121212116	3037	3861
6	Romance	735	2947	522.461872909699	2919	3620
7	Adventure	613	2463	547.9900793650794	2408	3080
8	Science Fiction	550	2242	582.2607655502393	2178	2782
9	Crime	528	2139	495.9339285714285	2013	2581
10	Family	524	2026	532.2789719626169	2138	2572

Figura 13.2: Resultado de la consulta de engagement por género en AWS Athena

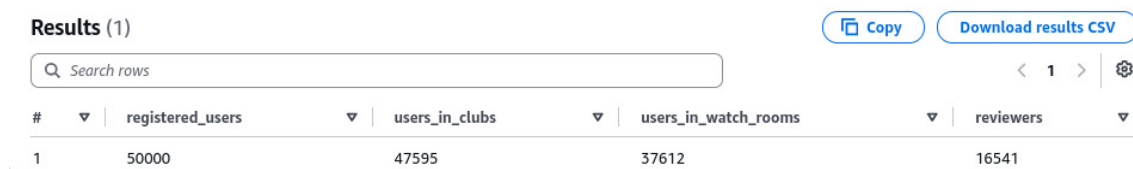
Consulta 2: Participación de usuarios en la plataforma

Para conocer qué tanto participan los usuarios, comparamos el total de registrados con quienes se han unido a clubes, participado en *watch rooms* o realizado reseñas. Se usan las tablas **users**, **memberships**, **watch_participants** e **interaction**. **COUNT(DISTINCT ...)** evita contar más de una vez a un usuario que participa en varias acciones.

```
SELECT
  (SELECT COUNT(1) FROM "astra_analytics"."users") AS registered_users ,
  (SELECT COUNT(DISTINCT user_id)
   FROM "astra_analytics"."memberships") AS users_in_clubs ,
  (SELECT COUNT(DISTINCT user_id)
   FROM "astra_analytics"."watch_participants") AS users_in_watch_rooms ,
  (SELECT COUNT(DISTINCT user_id)
   FROM "astra_analytics"."interaction"
   WHERE partition_0 = 'reviews') AS reviewers;
```

Listing 13.2: Consulta SQL de participación de usuarios en Athena

Como se observa en la Figura 13.3, Athena devolvió 50 000 usuarios registrados, 47 595 usuarios en clubes, 37 612 participantes en *watch rooms* y 16 541 revisores.



The screenshot shows the AWS Athena interface with a query result table. At the top, there are buttons for 'Copy' and 'Download results CSV'. Below the buttons is a search bar and navigation controls. The table has five columns: '#', 'registered_users', 'users_in_clubs', 'users_in_watch_rooms', and 'reviewers'. The first row of data shows the counts for each category.

#	registered_users	users_in_clubs	users_in_watch_rooms	reviewers
1	50000	47595	37612	16541

Figura 13.3: Resultado de la consulta de participación de usuarios en AWS Athena

Consulta 3: Métricas de interacción por película

Para revisar la actividad de cada película, unimos el catálogo, las interacciones y las salas de visualización. La consulta usa **movie**, **interaction** y **watch_rooms**, e incluye el identificador público, el título, el rating del catálogo, el promedio de puntuaciones, los likes y la cantidad de salas.

```
WITH movie_metrics AS (
  SELECT m.public_id, m.title, m.rating AS catalog_rating,
         (SELECT AVG(CAST(score AS DOUBLE))
          FROM "astra_analytics"."interaction"
          WHERE movie_id = CAST(m.public_id AS VARCHAR)
              AND partition_0 = 'reviews') AS avg_user_review_score,
         (SELECT COUNT(1) FROM "astra_analytics"."interaction"
          WHERE movie_id = CAST(m.public_id AS VARCHAR)
              AND partition_0 = 'likes') AS likes_count,
         (SELECT COUNT(1) FROM "astra_analytics"."watch_rooms"
          WHERE movie_id = CAST(m.public_id AS VARCHAR)) AS watch_room_count
  FROM "astra_analytics"."movie" m
)
SELECT * FROM movie_metrics
ORDER BY watch_room_count DESC
LIMIT 100;
```

Listing 13.3: Consulta SQL de métricas de interacción por película en Athena

La consulta ordena las películas por cantidad de salas y muestra las primeras 100 filas. El resultado de la Figura 13.4 incluye títulos como *Contracted: Phase II*, *Getting Even with Dad* y *Rise of the Guardians*, junto con sus identificadores públicos y métricas.

Completed

Time in queue: 83 ms

Run time: 2.082 sec

Data scanned: 15.94 MB

Results (100)

Copy

Download results CSV

Search rows

< 1 ... > ⚙

# ▾	public_id ▾	title ▾	catalog_rating ▾	avg_user_review_score ▾
1	fce67fb3-d5c8-42d0-9f23-4edaa2ab7d7b	Contracted: Phase II	2015.0	
2	0a4dfcda-382f-4705-9110-bc2a6d85a77f	Getting Even with Dad		
3	e57e54de-05df-4db3-974f-0aecc399d6b6	Rise of the Guardians		
4	417a9ede-2675-4f9c-abd2-0e7192066d89	Miss Austen Regrets	90.0	
5	f4f2b3fa-9de1-4d57-8536-edfa48d5e09f	Compliance		
6	8260ee0c-3076-474b-9e50-9a77883194f5	Vanishing on 7th Street	2010.0	
7	fb3bc234-fc51-48dc-9a80-af621efdeb26	Jack of the Red Hearts	7.0	
8	1edab6bb-813b-48e7-9c65-ada51a7057d6	Back in Time		
9	bac4527e-1057-4097-8343-d7c2e2b4ebd2	Sicario		
10	591625c8-b1da-4372-8866-6b7da86dc745	Excision	2012.0	
11	868cf08a-1d4a-4b56-b80a-09fb55deb9ff	Sorority Boys	93.0	

Figura 13.4: Resultado de la consulta de métricas de interacción por película en AWS Athena

Consulta 4: Nivel de actividad de los clubes

Para revisar la actividad de los clubes, unimos cada club con sus miembros, las salas creadas y las películas visualizadas. La consulta usa `clubs`, `memberships` y `watch_rooms`; muestra el nombre del club, sus miembros únicos, salas, relación de actividad y películas diferentes.

```
SELECT c.name AS club_name,
       COUNT(DISTINCT m.user_id) AS member_count,
       COUNT(DISTINCT wr.id) AS total_watch_rooms,
       CAST(COUNT(DISTINCT wr.id) AS DOUBLE)
       / NULLIF(COUNT(DISTINCT m.user_id), 0) AS activity_ratio,
       COUNT(DISTINCT wr.movie_id) AS unique_movies_watched
FROM "astra_analytics"."clubs" c
LEFT JOIN "astra_analytics"."memberships" m ON c.id = m.club_id
LEFT JOIN "astra_analytics"."watch_rooms" wr ON c.id = wr.club_id
GROUP BY c.name
ORDER BY activity_ratio DESC NULLS LAST;
```

Listing 13.4: Consulta SQL de nivel de actividad de clubes en Athena

Los `LEFT JOIN` incluyen también a los clubes que todavía no tienen membresías o salas. `NULLIF` evita una división entre cero cuando un club no registra miembros. La Figura 13.5 muestra 1 712 resultados ordenados por `activity_ratio`. Townsend-Wood Club 1456 registra 53 miembros, 11 salas y una relación de actividad de aproximadamente 0.208; Huerta-Wallace Club 307 registra 79 miembros y 13 salas.

Completed Time in queue: 112 ms Run time: 2.201 sec Data scanned: 10.07 MB						
Results (1,712) Copy Download results CSV						
Search rows < 1 ... >						
#	club_name	member_count	total_watch_rooms	activity_ratio	unique_movies_watched	
1	Townsend-Wood Club 1456	53	11	0.20754716981132076	11	
2	"Conley	66	12	0.18181818181818182	12	
3	Parker Inc Club 14	62	11	0.1774193548387097	11	
4	Willis Group Club 1497	58	10	0.1724137931034483	10	
5	"Mayer	65	11	0.16923076923076924	11	
6	Huerta-Wallace Club 307	79	13	0.16455696202531644	13	
7	Alexander-Moses Club 1576	68	11	0.16176470588235295	11	
8	Mcmillan PLC Club 1215	68	11	0.16176470588235295	11	
9	Keller-Mcdonald Club 1213	62	10	0.16129032258064516	10	
10	Martinez-Bowman Club 350	63	10	0.15873015873015872	10	
11	Huerta-Wallace Club 307	79	13	0.16455696202531644	13	

Figura 13.5: Resultado de la consulta de actividad de clubes en AWS Athena

13.3 Vistas analíticas en AWS Athena

Además de las consultas, se crearon vistas en Athena para reutilizar datos de distintos componentes de ASTRA CLOUD.

Vista 1: Consolidado analítico de películas

La vista `vw_consolidado_peliculas` une el catálogo, sus géneros, las salas de visualización y las interacciones. Usa `movie`, `movie_genre`, `genre`, `watch_rooms` e `interaction` para mostrar el identificador de película, el título, el género, el total de salas, los likes y el promedio de puntuaciones.

```
CREATE OR REPLACE VIEW vw_consolidado_peliculas AS
SELECT m.public_id AS movie_id, m.title, g.name AS genre,
       COUNT(DISTINCT w.id) AS total_salas,
       COUNT(i_likes.user_id) AS total_likes,
       AVG(CAST(i_reviews.score AS DOUBLE)) AS promedio_usuarios
FROM "astra_analytics"."movie" m
LEFT JOIN "astra_analytics"."movie_genre" mg ON m.id = mg.movie_id
LEFT JOIN "astra_analytics"."genre" g ON mg.genre_id = g.id
LEFT JOIN "astra_analytics"."watch_rooms" w
  ON CAST(m.public_id AS VARCHAR) = w.movie_id
LEFT JOIN "astra_analytics"."interaction" i_likes
  ON CAST(m.public_id AS VARCHAR) = i_likes.movie_id
  AND i_likes.partition_0 = 'likes'
LEFT JOIN "astra_analytics"."interaction" i_reviews
  ON CAST(m.public_id AS VARCHAR) = i_reviews.movie_id
  AND i_reviews.partition_0 = 'reviews'
GROUP BY m.public_id, m.title, g.name;
```

Listing 13.5: Creación de la vista `vw_consolidado_peliculas`

La Figura 13.6 muestra la creación de la vista y la Figura 13.7, su consulta. Entre las filas aparecen *Jurassic World*, *Mad Max: Fury Road* y *Terminator Genisys*, con sus géneros, salas y likes. Con esta vista no es necesario repetir las mismas uniones en cada consulta.



Figura 13.6: Creación de la vista vw_consolidado_peliculas en AWS Athena

#	movie_id	title	genre	total_salas	total_likes
1	fdcd1319-aa5f-4d21-807f-9f32b1766804	Jurassic World	Science Fiction	4	20
2	ecb6a464-9acf-44d8-9af4-66e62a425052	Mad Max: Fury Road	Science Fiction	5	45
3	89339279-ceb3-412b-ae0c-55fdb40893ad	Terminator Genisys	Thriller	3	66

Figura 13.7: Consulta de la vista vw_consolidado_peliculas en AWS Athena

Vista 2: Perfil social de usuarios

La vista vw_perfil_social_usuarios reúne la participación social de cada usuario. Relaciona users, memberships y watch_participants para mostrar el identificador, username, correo electrónico, clubes unidos y salas en las que participó.

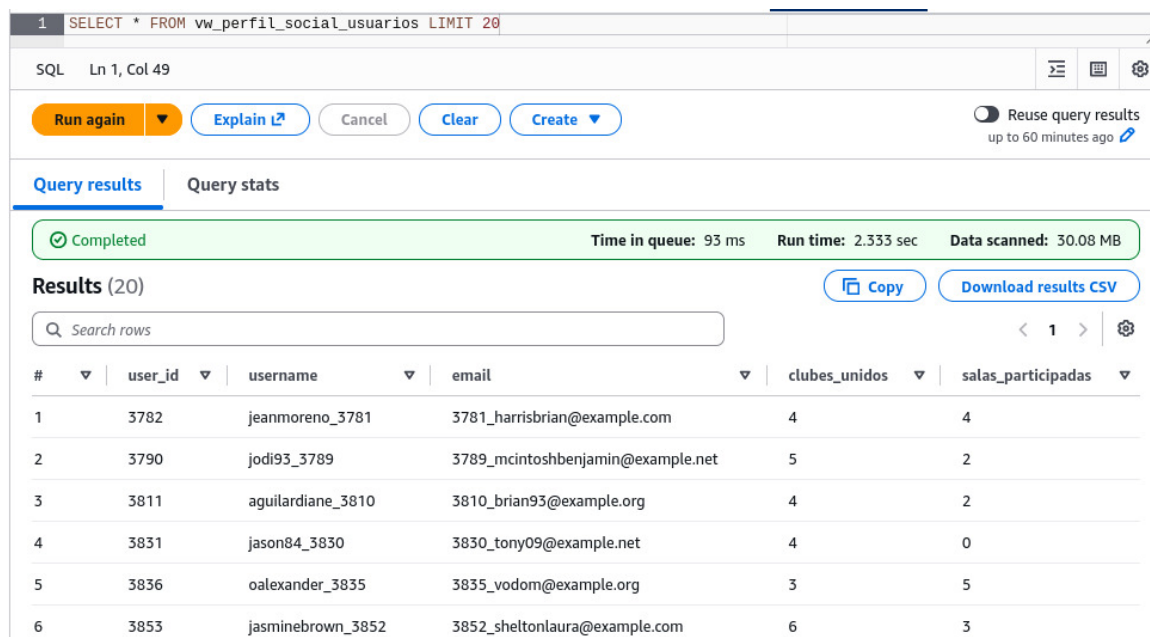
```
CREATE OR REPLACE VIEW vw_perfil_social_usuarios AS
SELECT u.id AS user_id, u.username, u.email,
       COUNT(DISTINCT m.club_id) AS clubes_unidos,
       COUNT(DISTINCT wp.watch_room_id) AS salas_participadas
FROM "astra_analytics"."users" u
LEFT JOIN "astra_analytics"."memberships" m
  ON CAST(u.id AS VARCHAR) = CAST(m.user_id AS VARCHAR)
LEFT JOIN "astra_analytics"."watch_participants" wp
  ON CAST(u.id AS VARCHAR) = CAST(wp.user_id AS VARCHAR)
GROUP BY u.id, u.username, u.email;
```

Listing 13.6: Creación de la vista vw_perfil_social_usuarios

La Figura 13.8 muestra la creación de la vista y la Figura 13.9, su consulta. Entre las filas aparece jeanmoreno_3781, con 4 clubes y 4 salas. Así se puede consultar rápidamente en cuántos clubes y salas participó cada usuario.



Figura 13.8: Creación de la vista vw_perfil_social_usuarios en AWS Athena



1 `SELECT * FROM vw_perfil_social_usuarios LIMIT 20`

SQL Ln 1, Col 49

[Run again](#) [Explain](#) [Cancel](#) [Clear](#) [Create](#) ☐ Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 93 ms Run time: 2.333 sec Data scanned: 30.08 MB

Results (20) [Copy](#) [Download results CSV](#)

Search rows

#	user_id	username	email	clubes_unidos	salas_participadas
1	3782	jeanmoreno_3781	3781_harrisbrian@example.com	4	4
2	3790	jodi93_3789	3789_mcintoshbenjamin@example.net	5	2
3	3811	aguilardiane_3810	3810_brian93@example.org	4	2
4	3831	jason84_3830	3830_tony09@example.net	4	0
5	3836	oalexander_3835	3835_vodom@example.org	3	5
6	3853	jasminebrown_3852	3852_sheltonlaura@example.com	6	3

Figura 13.9: Consulta de la vista vw_perfil_social_usuarios en AWS Athena

14 Seguridad

14.1 Identidad y acceso

Identity se encarga del registro y del inicio de sesión. Las contraseñas no se guardan directamente, sino como hashes. Al iniciar sesión, el servicio emite un JWT HS256. Community, Interaction y Cinema Session revisan ese token antes de atender rutas protegidas y no almacenan contraseñas.

14.2 Red y configuración

La plantilla define grupos de seguridad para el ALB, las instancias de producción y las bases de datos. PostgreSQL, MySQL y MongoDB solo aceptan conexiones desde el grupo de producción. Los secretos y contraseñas se reciben como parámetros NoEcho y variables de entorno. El bucket de pósters usa cifrado SSE-S3.

14.3 Consideraciones

Para el entorno académico se dejaron subredes públicas, SSH abierto y lectura pública solo para el bucket de pósters. En producción se debería restringir SSH y CORS, usar TLS y guardar los secretos con Secrets Manager o SSM.

15 Infraestructura definida como código

La infraestructura de ASTRA CLOUD se describe con AWS CloudFormation en `../infrastructure/aws/proycloud-infrastructure.yaml`. Allí se definen la red, los grupos de seguridad, las instancias EC2, el DNS privado, el ALB, los grupos destino, API Gateway con VPC Link y el bucket de medios. Los bloques `UserData` preparan las instancias con Docker y Docker Compose. De esta forma, la configuración necesaria para desplegar la solución queda guardada en el proyecto.

16 Flujos funcionales

16.1 Autenticación y comunidad

El cliente registra una cuenta o inicia sesión en Identity y recibe un JWT. Community revisa el token y asocia el perfil local con el UUID `sub`. Después, el usuario puede crear clubes, unirse a ellos y abrir salas.

16.2 Catálogo e interacciones

Catalog entrega las películas y sus datos. Cuando un usuario crea una reseña, like, watchlist o visualización, Interaction revisa el token, consulta los servicios necesarios y guarda la acción junto con los datos relacionados.

16.3 Sesión y analítica

Cinema Session coordina una sesión temporal y envía los eventos por WebSocket. Los contenedores de carga extraen las tres bases de datos y suben archivos CSV a S3. Después, Analytics consulta Athena para obtener los indicadores.

17 Pruebas y validación

El proyecto incluye pruebas automatizadas para Community, Interaction y Cinema Session. Community y Cinema Session usan Pytest para revisar rutas, permisos y flujos de salas. Interaction usa `node:test`, Supertest y `mongodb-memory-server`, por lo que puede probar sus rutas sin una instancia externa de MongoDB. Los servicios también tienen rutas de salud que usan la integración y el balanceo. Interaction incluye `npm run seed`, un script que puede ejecutarse varias veces sin duplicar los datos que ya existen. Por defecto genera 20 000 documentos para reseñas, watchlists e historial, y 25 000 para likes. Las variables `SEED_*` permiten ajustar estos escenarios.

18 Consideraciones de costos

El costo depende principalmente de las cuatro instancias EC2 `t3.medium`, EBS, ALB, API Gateway, S3 y el volumen de datos que procesa Athena. Glue y la transferencia de datos también pueden generar cargos cuando se habiliten. Como Athena consulta los archivos cuando se necesita, no es necesario mantener un clúster analítico permanente.

19 Limitaciones actuales

- Cinema Session guarda el estado en memoria local. Para compartirlo entre más réplicas se necesitaría un servicio como Redis.
- JWT HS256 usa un secreto común en los servicios que validan los tokens.
- La plantilla no incluye un Auto Scaling Group ni alta disponibilidad para la instancia de base de datos.
- El proceso genera y carga archivos CSV, pero la frecuencia de ejecución y los recursos de Glue se deben configurar al desplegarlo.
- El código fuente del frontend y la configuración de AWS Amplify no están incluidos en este proyecto, aunque la aplicación web desplegada sigue disponible.

20 Mejoras futuras

Como mejoras futuras, se podría agregar Redis para compartir el estado de Cinema Session, un servicio administrado de secretos, monitoreo y alertas con CloudWatch, respaldos, Auto

Scaling Groups y CI/CD con pruebas y análisis de imágenes. En la parte analítica, los procesos de Glue que detectan la estructura de los archivos, las particiones y una programación de cargas permitirían ejecutar el proceso de forma periódica.

21 Conclusiones

ASTRA CLOUD permitió implementar una plataforma cinematográfica organizada en seis microservicios con responsabilidades diferenciadas. Identity, Catalog, Community, Interaction, Cinema Session y Analytics colaboran mediante interfaces definidas, lo que mantiene separadas la autenticación, el catálogo, la interacción social, las sesiones compartidas y la consulta analítica.

La solución adopta persistencia políglota: PostgreSQL respalda las cuentas y el catálogo, MySQL concentra la información comunitaria y MongoDB conserva las interacciones. Esta elección responde al tipo de información administrada por cada servicio y se complementa con Docker y Docker Compose para reproducir los entornos de ejecución.

El despliegue se describe con CloudFormation e integra VPC, instancias EC2, API Gateway, VPC Link, ALB y S3. Además, el flujo PostgreSQL/MySQL/MongoDB → CSV → S3 → Athena permite analizar los datos extraídos sin trasladar esa carga a las bases operativas. En conjunto, el proyecto demuestra la integración de servicios, persistencia, despliegue cloud y analítica dentro de una arquitectura coherente para el alcance académico planteado.

22 ANEXOS

22.1 Anexo A. Repositorios del proyecto

La Tabla 22.1 reúne los repositorios cuyos remotos Git fueron verificados en los componentes disponibles del proyecto.

Cuadro 22.1: Repositorios públicos verificados de ASTRA CLOUD.

Componente	Tecnología	Repositorio GitHub
Identity Service	Python, FastAPI	https://github.com/dado4cs/identity-service
Catalog Service	Java, Spring Boot	https://github.com/Pepe17xd/backend_catalogo
Community Service	Python, FastAPI	https://github.com/Pepe17xd/backend_community
Interaction Service	Node.js, Express	https://github.com/jmtimana/interaction-service
Cinema Session Service	Python, FastAPI	https://github.com/Jhynfe/cinema-session-service
Analytics Service	Python, FastAPI	https://github.com/dado4cs/data-analitica-ms-cloud

22.2 Anexo B. Aplicación web desplegada

La siguiente dirección corresponde a la aplicación web oficial de ASTRA CLOUD desplegada en AWS Amplify:

<https://feature-http-polling-chat.dgg2xrxydvbjy.amplifyapp.com/>

Bibliografía

- [1] Amazon Web Services. Amazon athena user guide, 2026.
- [2] Docker Inc. Docker documentation, 2026.
- [3] FastAPI. Fastapi documentation, 2026.
- [4] Mongoose. Mongoose documentation, 2026.
- [5] PostgreSQL Global Development Group. Postgresql documentation, 2026.
- [6] Proyecto ASTRA CLOUD. Astra cloud source repositories, 2026.
- [7] VMware. Spring boot reference documentation, 2026.